FOUNDATIONS OF THINKING

How Engineers Think Before Coding

Computational + Meta Thinking

Presenter: Adeel Atta

Question

You have a 1000+ page dictionary.

Find the word:

“ Information ”

In how many different possible ways can you search for the word?

Computational Thinking

Structured way to solve problems logically and efficiently

Key Elements

- Decomposition
- Pattern Recognition
- Abstraction
- Algorithmic Formulation

Decomposition

Breaking a big problem into smaller parts until it feels easy.(or at least manageable)

Example: Food Delivery App

- Secure login system
- Restaurant listing
- Cart
- Payment
- Delivery tracking

Decomposition

Decomposition Example

Check if a password is valid.

Rules:

- At least 8 characters
- Contains digit
- Contains uppercase letter

Decompose into:

- Check length
- Check digit existence
- Check uppercase existence

Pattern Recognition

Identify similarities or structure

What is the pattern in these examples

2, 4, 6, 8, 10, ?

Red → Green → Yellow → Red → ??

1, 4, 9, 16, 25, ?

1, 1, 2, 3, 5, 8, 13, ?

Abstraction

Focus on important details

Ignore unnecessary details

When Driving a car

We use:

- Steering
- Brake
- Accelerator

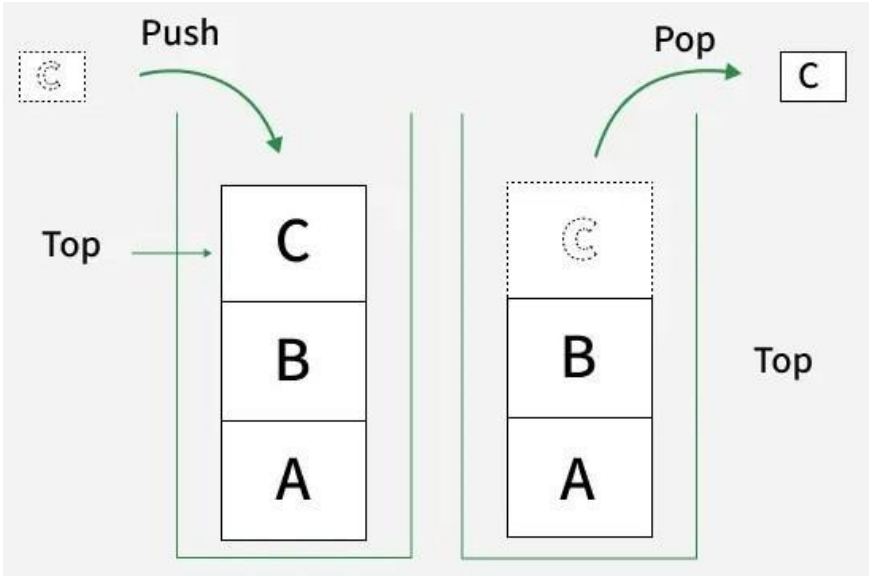
We don’t think about:

- Engine combustion
- Piston movement
- Tyre Rotations

Abstraction in DSA

Stack operations:

- push()
- pop()



We don’t care:

If internally it uses array or linked list

We care only about behavior

Abstraction allows us to build complex systems without knowing every internal detail

Algorithmic Formulation

A step-by-step logical procedure to solve a problem.

Example: Making tea

1. Boil water
2. Add tea
3. Add sugar & milk
4. Serve

Algorithm Example – Maximum

Find max in: [8, 3, 10, 5, 6]

1. Assume first is max
2. Compare with others
3. Update when larger
4. Return max

Why These Skills Matter

Every DSA problem requires:

- Decomposition
- Pattern Recognition
- Abstraction
- Algorithm Design

Strong programmers think first, type later.

Summary

Decomposition → Break problem into parts

Pattern Recognition → Identify structure

Abstraction → Focus on essential behavior

Algorithmic Formulation → Step-by-step logic

Meta Thinking

Thinking about thinking

Improving how we solve problems

Ask better questions

Q1 # Minimal Information Needed

What is essential?

What can be ignored?

Remove unnecessary details

Example: Identifying Noise in a Problem

Ali is 20 years old.

- He likes cricket.
- He lives in Lahore.

Question: When will he turn 25?

Essential info: Current age + years to add

Q2 # Can I Compress the State?

Can we store less information?

- Represent data efficiently
- Simplify variables

Compress State Example

Instead of keeping all the extra details we only store the numbers we need.

Ali is 20 years old.
- He likes cricket.
- He lives in Lahore.

Question: When will he turn 25?

Full information stored:
- Age = 20
- Hobbies = cricket
- City = Lahore
- Favorite color = blue

Compressed state:

current_age = 20

years_to_add = 5

What is Invariant?

What never changes?

- Find constant rules
- Use invariants to prove correctness

Invariant Example

Instead of keeping all the extra details we only store the numbers we need.

Ali is 20 years old.
- He likes cricket.
- He lives in Lahore.

Question: When will he turn 25?

While calculating Ali's future age:

Rule: future_age = current_age + years_to_add

Invariant: The difference future_age - current_age = years_to_add always stays true, no matter what the values are.

Applying All Three Together

Designing a Tournament Scheduler

Minimal info: Teams, venues, dates

Compressed state: Match count

Invariant: No team plays twice at same time